

Norman's design principles

Slides by Kiran Khurshid

Norman's design principles

1. Visibility
2. Feedback
3. Constraints
4. Mapping
5. Consistency
6. Affordance

Visibility

The more visible functions are, the more likely users will be able to know what to do next. In contrast, when functions are out of sight, it makes them more difficult to find and know how to use.

The skill in applying this principle is realizing that you can't make everything visible, because it'll ultimately clutter the interface but instead need to prioritize what interface elements are by far the most important for the user experience and prioritize their visibility.

Visibility - Example

Hamburger side-bar menu

Vs.

Tab-bar menu



Feedback

Feedback is about sending back information about what action has been done and what has been accomplished, allowing the person to continue with the activity. Various kinds of feedback are available for interaction design-audio, tactile, verbal, and combinations of these.

Feedback - Example

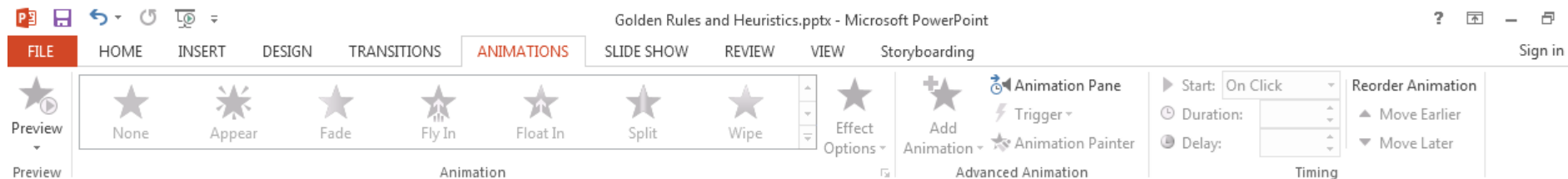
Here's a brief video showing the many ways that Google's Material design pattern encourages you to provide feedback to the user on the actions they are taking in a mobile app: <https://youtu.be/Q8TXgCzxEnw>

Constraints

The design concept of constraining refers to determining ways of restricting the kind of user interaction that can take place at a given moment. There are various ways this can be achieved.

Constraints is about limiting the range of interaction possibilities for the user to simplify the interface and guide the user to the appropriate next action. This is a case where constraints are clarifying, since they make it clear what can be done. Limitless possibilities often leave the user confused

Constraints - Example

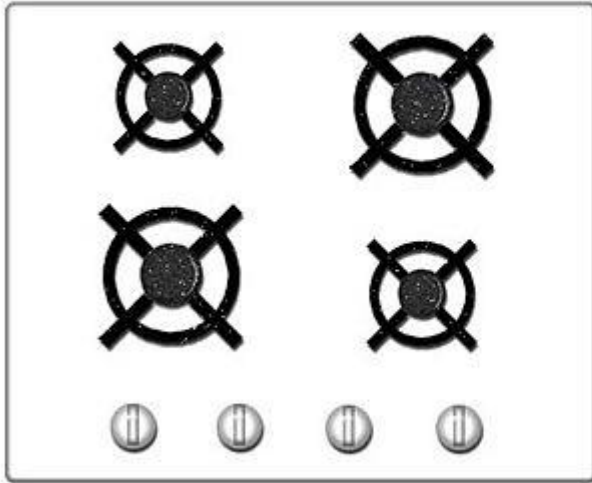
[illegible]

Mapping

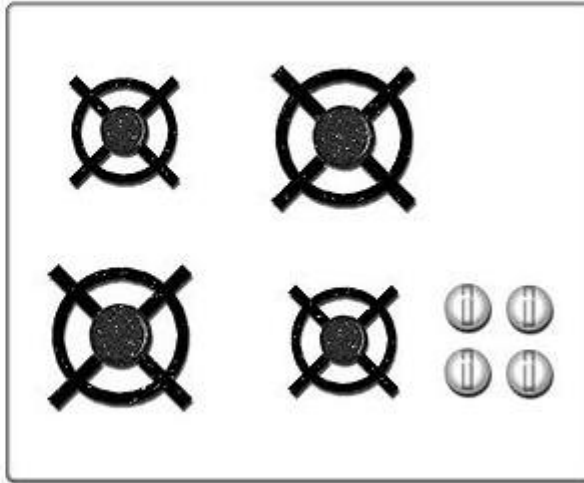
This refers to the relationship between controls and their effects in the world. Nearly all artifacts need some kind of mapping between controls and effects, whether it is a flashlight, car, power plant, or cockpit. An example of a good mapping between control and effect is the up and down arrows used to represent the up and down movement of the cursor, respectively, on a computer keyboard.

Mappings - Example

Poor mapping

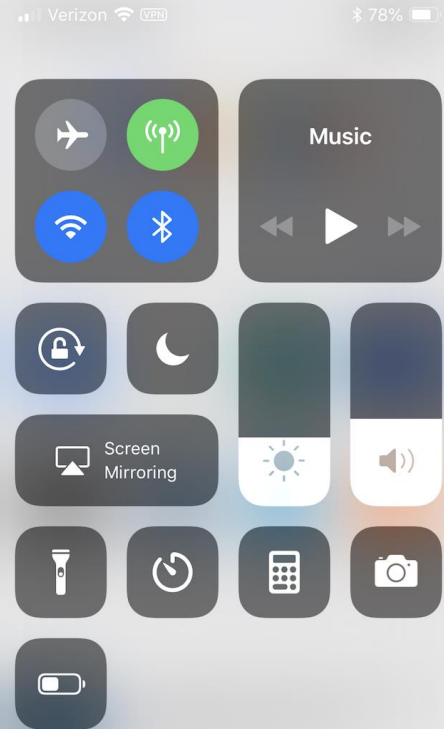


Good mapping



So, put Caps Lock light
where Caps Lock button
is!

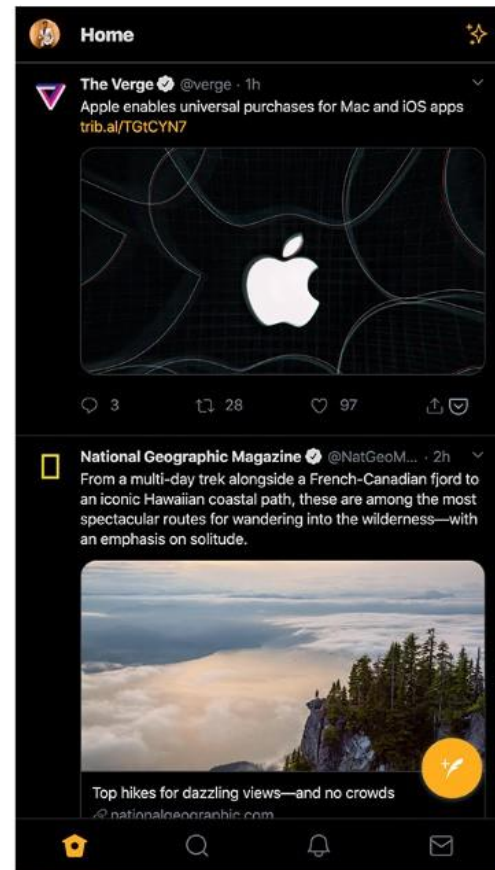
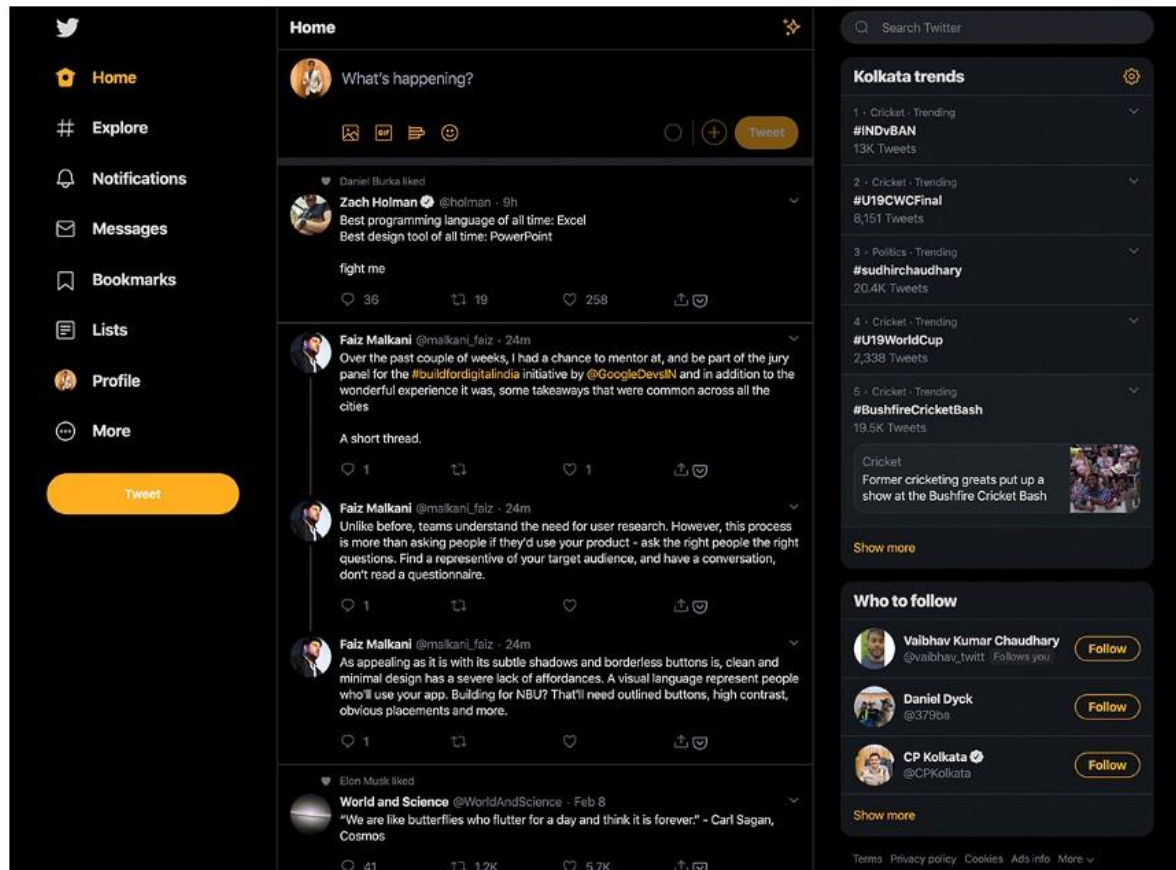
▲ If you have to label it,
reconsider your design



Consistency

This refers to designing interfaces to have similar operations and use similar elements for achieving similar tasks. In particular, a consistent interface is one that follows rules, such as using the same operation to select all objects. For example, a consistent operation is using the same input action to highlight any graphical object at the interface, such as always clicking the left mouse button.

Consistency - Example



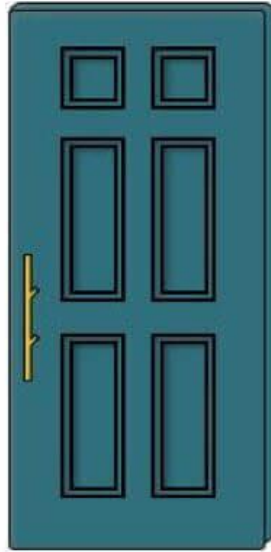
Affordance

A term used to refer to an attribute of an object that allows people to know how to use it. For example, a mouse button invites pushing (in so doing acting clicking) by the way it is physically constrained in its plastic shell. At a very simple level, to afford means to give a clue (Norman, 1988). When the affordances of a physical object are perceptually obvious it is easy to know how to interact with it.

Affordance - Example



↻ TURN



↓ PULL



↑ PUSH

Affordances - Provide clues

As web and mobile design has been evolving to more modern interfaces, in some ways we have been removing many of the common affordances we have classically used. For example, **the blue underlined link** used to be the strongest affordance for knowing something was a clickable link on the web. Many web experiences have moved away from this in an attempt to allow link elements to better match with the overall design aesthetic they are trying to achieve across the interface. While that's a helpful goal, we have to ensure that it's still clear what in fact are clickable links on the page using alternative affordances, like showing the underline on hover. When we remove all such affordances, the interface becomes meaningfully harder to use.